Chapter 18 (Paging: Introduction) — Homework (Simulation)

Zhiyuan Pan

All runs use `paging-linear-translate.py` in `ostep-homework/vm-paging/`. Every command below can be rerun with `code/chapter18/run.sh`. For counting page-table entries and averaging over many seeds I used a small helper, `code/chapter18/paging_stats.py`; it runs the official simulator and only counts its output.

How the simulator translates an address:

- The page table is a **linear array** with one 4-byte entry per virtual page. The top bit is the **valid** bit; the rest of a valid entry is the **PFN**.
- With an address space of size A and page size P, a VA has log2(A) bits: the low log2(P) bits are the **offset**, the rest is the **VPN**.
- **PA = (PFN << log2(P)) | offset**, or the access is invalid if the entry's valid bit is 0.

Q1

> Before doing any translations, let's use the simulator to study how linear page tables change size given different parameters. Compute the size of linear page tables as different parameters change. Some suggested inputs are below; by using the `-v` flag, you can see how many page-table entries are filled. First, to understand how linear page table size changes as the address space grows, run with these flags: `-P 1k -a 1m -p 512m -v -n 0`, `-P 1k -a 2m -p 512m -v -n 0`, `-P 1k -a 4m -p 512m -v -n 0`. Then, to understand how linear page table size changes as page size grows: `-P 1k -a 1m -p 512m -v -n 0`, `-P 2k -a 1m -p 512m -v -n 0`, `-P 4k -a 1m -p 512m -v -n 0`. Before running any of these, try to think about the expected trends. How should page-table size change as the address space grows? As the page size grows? Why not use big pages in general?

Expected trend. A linear page table has one entry per virtual page, so

$$\text{entries} = \frac{\text{address-space size}}{\text{page size}}, \qquad \text{table size} = \text{entries} \times 4 \text{ bytes}.$$

It grows **linearly with the address space** and **shrinks in proportion as the page size grows**. The physical memory size does not matter. The runs agree:

flags	entries	table size	valid entries (`-u 50`)
`-P 1k -a 1m`	1m / 1k = **1024**	4 KB	519
`-P 1k -a 2m`	**2048**	8 KB	1035
`-P 1k -a 4m`	**4096**	16 KB	2062
`-P 1k -a 1m`	**1024**	4 KB	519
`-P 2k -a 1m`	**512**	2 KB	259
`-P 4k -a 1m`	**256**	1 KB	133

Doubling the address space doubles the table. Doubling the page size halves it. Note that the table always has an entry for **every** virtual page, even though only about half are valid.

Why not use big pages in general? Big pages make the table small, but they waste memory through **internal fragmentation**. Memory is handed out in whole pages, so a process that needs a few bytes of stack or heap still gets an entire page; with 1 MB pages most of each page would be unused. Big pages also make paging to disk and copy-on-write more expensive (a whole large page has to be moved or copied), and the OS can manage memory less finely. That is why systems use small pages (4 KB) by default, add large pages (2 MB / 1 GB) only as an option, and attack the page-table size problem in other ways (multi-level tables, chapter 20).

Q2

> Now let's do some translations. Start with some small examples, and change the number of pages that are allocated to the address space with the `-u` flag. For example: `-P 1k -a 16k -p 32k -v -u 0`, `-u 25`, `-u 50`, `-u 75`, `-u 100`. What happens as you increase the percentage of pages that are allocated in each address space?

With `-P 1k -a 16k`, a VA has 14 bits: **4 bits of VPN** (16 pages) and **10 bits of offset**. So VPN = VA >> 10, offset = VA & 0x3ff, and PA = (PFN << 10) | offset.

`-u 0`: all 16 entries are invalid, so every address is invalid: 0x3a39 (VPN 14), 0x3ee5 (VPN 15), 0x33da (VPN 12), 0x39bd (VPN 14), 0x13d9 (VPN 4).

`-u 25`: 6 of 16 valid (VPN 0→0x18, 5→0x09, 8→0x10, 10→0x13, 12→0x1f, 13→0x1c).

VA	VPN, offset	entry	PA
0x3986	14, 0x186	invalid	**invalid**
0x2bc6	10, 0x3c6	PFN 0x13	0x13 << 10 = 0x4c00, + 0x3c6 = **0x4fc6** (20422)
0x1e37	7, 0x237	invalid	**invalid**
0x0671	1, 0x271	invalid	**invalid**
0x1bc9	6, 0x3c9	invalid	**invalid**

`-u 50`: 9 of 16 valid (VPN 0→0x18, 3→0x0c, 4→0x09, 6→0x1d, 7→0x13, 9→0x1f, 10→0x1c, 12→0x0f, 15→0x08).

VA	VPN, offset	entry	PA
0x3385	12, 0x385	PFN 0x0f	0x3c00 + 0x385 = **0x3f85** (16261)
0x231d	8, 0x31d	invalid	**invalid**
0x00e6	0, 0x0e6	PFN 0x18	0x6000 + 0x0e6 = **0x60e6** (24806)
0x2e0f	11, 0x20f	invalid	**invalid**
0x1986	6, 0x186	PFN 0x1d	0x7400 + 0x186 = **0x7586** (30086)

`-u 75` and `-u 100`: with seed 0 all 16 entries happen to be valid in both runs, and the two page tables are identical (VPN 0→0x18, 6→0x1f, 10→0x03, 11→0x13, 13→0x1b, ...), so the translations are the same:

VA	VPN, offset	entry	PA
0x2e0f	11, 0x20f	PFN 0x13	0x4c00 + 0x20f = **0x4e0f** (19983)
0x1986	6, 0x186	PFN 0x1f	0x7c00 + 0x186 = **0x7d86** (32134)
0x34ca	13, 0x0ca	PFN 0x1b	0x6c00 + 0x0ca = **0x6cca** (27850)
0x2ac3	10, 0x2c3	PFN 0x03	0x0c00 + 0x2c3 = **0x0ec3** (3779)
0x0012	0, 0x012	PFN 0x18	0x6000 + 0x012 = **0x6012** (24594)

All my translations matched `-c`.

What happens as `-u` grows: more page-table entries are valid, so more of the virtual addresses translate successfully and fewer are invalid (0, 1, 3, 5, 5 out of 5 here). Averaged over 200 seeds, `-u 25 / 50 / 75` gives 24.7% / 48.0% / 71.0% valid entries, slightly below `-u` because the simulator re-draws (and may then mark the page invalid) when it picks a physical frame that is already taken. Also, a larger `-u` uses more physical frames (at `-u 100`, 16 of the 32 frames).

The important thing that does **not** change is the page table itself: it is **16 entries (64 bytes) in every case**, even at `-u 0`, when the process uses no memory at all. A linear page table costs memory in

proportion to the size of the address space, not to how much of it is actually used. That is the main weakness of linear page tables.

Q3

> Now let's try some different random seeds, and some different (and sometimes quite crazy) address-space parameters, for variety: `-P 8 -a 32 -p 1024 -v -s 1`, `-P 8k -a 32k -p 1m -v -s 2`, `-P 1m -a 256m -p 512m -v -s 3`. Which of these parameter combinations are unrealistic? Why?

`-P 8 -a 32 -p 1024 -s 1`: 5-bit VA = 2-bit VPN + 3-bit offset. The page table is [invalid, PFN 0x61, invalid, invalid].

VA	VPN, offset	PA
14 (0x0e)	1, 6	0x61 × 8 + 6 = **782** (0x30e)
20 (0x14)	2, 4	**invalid**
25 (0x19)	3, 1	**invalid**
3 (0x03)	0, 3	**invalid**
0 (0x00)	0, 0	**invalid**

`-P 8k -a 32k -p 1m -s 2`: 15-bit VA = 2-bit VPN + 13-bit offset. The table is [PFN 0x79, invalid, invalid, PFN 0x5e]. All five addresses fall on VPN 1 or 2 (0x55b9 → 2, 0x2771 → 1, 0x4d8f → 2, 0x4dab → 2, 0x4a64 → 2), so **all five are invalid**.

`-P 1m -a 256m -p 512m -s 3`: 28-bit VA = 8-bit VPN + 20-bit offset; the table has 256 entries, 120 valid. For example, VA 0x0308b24d is VPN 0x30 = 48, offset 0x8b24d; entry 48 holds PFN 0x1f6, so the PA is **0x1f68b24d** (526955085). The other answers: 0x042351e6 invalid (VPN 66), 0x02feb67b → 0x0a9eb67b (VPN 47), 0x0b46977d invalid (VPN 180), 0x0dbcceb4 → 0x1f2cceb4 (VPN 219).

Which are unrealistic?

- `-P 8 -a 32` **is clearly unrealistic.** An 8-byte page holds two page-table entries; the 4-entry page table (16 bytes) is half the size of the whole 32-byte address space. Every few bytes of memory would need its own translation, so the translation overhead would be enormous. No real machine has such small pages or address spaces.
- `-P 8k -a 32k -p 1m`: 8 KB pages are realistic (some systems use them), but a 32 KB address space and 1 MB of physical memory are toy sizes. The proportions are fine; it is just a scaled-down system.
- `-P 1m -a 256m -p 512m` is the closest to a real system. 1 MB pages are larger than the usual 4 KB, but real CPUs do support large pages (2 MB on x86-64). Its weakness is internal fragmentation: every partly used region wastes up to 1 MB.

More generally, all of these runs are unrealistic in one way: the simulator requires **physical memory to be larger than the address space**. On real machines it is the other way around: a 64-bit (or even 32-bit) address space is much bigger than physical memory, and the OS relies on most of it being unused or swapped out (chapter 21).

Q4

> Use the program to try out some other problems. Can you find the limits of where the program doesn't work anymore? For example, what happens if the address-space size is bigger than physical memory?

what I tried	result
`-a 32k -p 32k` or `-a 64k -p 32k` (address space ≥ physical memory)	`Error: physical memory size must be GREATER than address space size (for this simulation)`
`-a 512m -p 1g`	`Error: must use smaller sizes (less than 1 GB)`: both sizes must be under 1 GB
`-P 1k -a 12k`	`address space must be a power of 2`
`-P 32k -a 16k` (page bigger than the address space)	`address space must be a multiple of the pagesize`
`-P 0`	**crash**: `ZeroDivisionError` (not checked)
`-A 20000` with `-a 16k` (a VA outside the address space)	**crash**: `IndexError: array index out of range`; the VPN is past the end of the table
`-u 150` / `-u -10`	no error: behaves like `-u 100` / `-u 0`
`-P 1 -a 16 -p 32`	works: 1-byte pages, 0 offset bits, PA = PFN
`-P 1 -a 1m -p 2m -n 0`	works, but the table has 1,048,576 entries and takes about 1 s; `-P 1 -a 256m` would need 268M entries

Address space bigger than physical memory: the simulator refuses it. A linear page table could describe such an address space: entries can simply be invalid. But the simulator wants to be able to map every virtual page to a distinct frame (it does so at `-u 100`), so it insists on physical memory being larger. A real OS handles this case by keeping only some pages in memory and the rest on disk (swapping, chapter 21), which the simulator does not model.

The other limits come from the simulator's design: sizes must be powers of two so the VPN/offset split is a bit split; the 1 GB limit keeps PFNs within the 31 bits left beside the valid bit and keeps the table small enough to print. Two inputs are simply not validated (`-P 0` and an out-of-range `-A` address), and the program crashes instead of reporting an error.